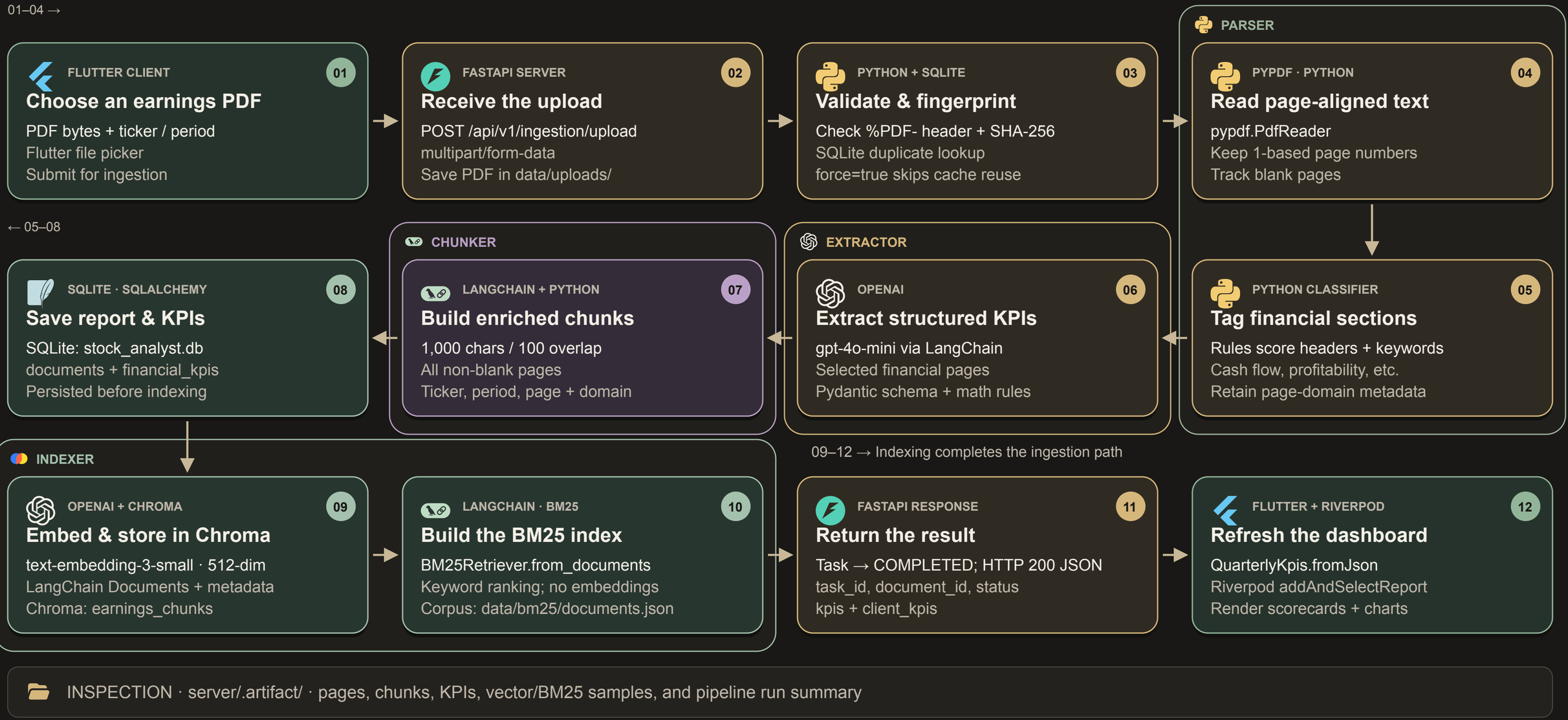
From earnings PDF to searchable evidence

Upload, parse, extract, chunk, persist, and build dense + sparse indexes before returning to Flutter.



What makes ingestion useful for RAG?

Keep small, traceable pieces of evidence that the vector and keyword indexes can search.

PARSER

PYPDF · PYTHON — 01

Keep page boundaries

pypdf extracts page text
Page numbers start at 1
Blank pages are skipped later

PYTHON CLASSIFIER — 02

Keep financial context

Rule-based domain labels
Headers + body keywords
Each page keeps its domain

EXTRACTOR

OPENAI · GPT-4O-MINI — 03

Resolve report identity

KPIs supply canonical metadata
Ticker + fiscal quarter + year
Statement pages sent to LLM

CHUNKER

LANGCHAIN — 04

Split within each page

RecursiveCharacterTextSplitter
1,000 characters per chunk
100-character overlap

PYTHON ENRICHMENT — 05

Attach source metadata

Stable chunk ID + text
Ticker, period, page + domain
Chunk position + char count

Chunking uses all non-blank pages, including narrative pages outside the KPI extraction subset.

A SAVED CHUNK · actual inspection-artifact metadata

```
chunk_id: "TSLA_Q2_2026_p3_c0"
ticker / period: "TSLA" / "Q2" / 2026
page_number: 3
financial_domain: "cash_flow"
chunk position: 0 of 4 · char_count: 943
```

WHERE THE EVIDENCE LIVES TODAY

Chunks feed Chroma + BM25

03b_enriched_chunks_*.json / .md

Written when save_artifacts=true

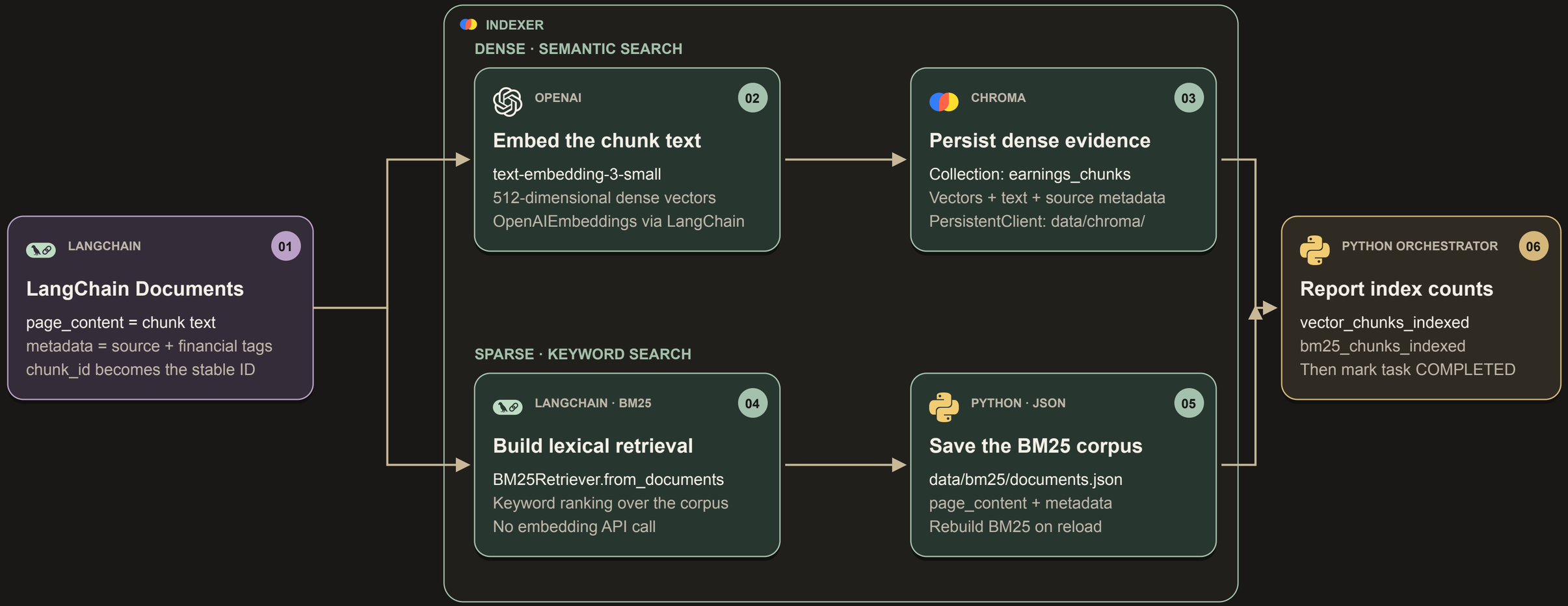
The Flutter upload enables artifact saving.

INDEXING HANDOFF · implemented in the ingestion orchestrator

Same enriched chunks → OpenAI embeddings + Chroma dense index, and LangChain BM25 sparse index

One chunk set, two searchable indexes

Dense vectors capture meaning. Sparse BM25 ranks keyword matches. Both retain source metadata.



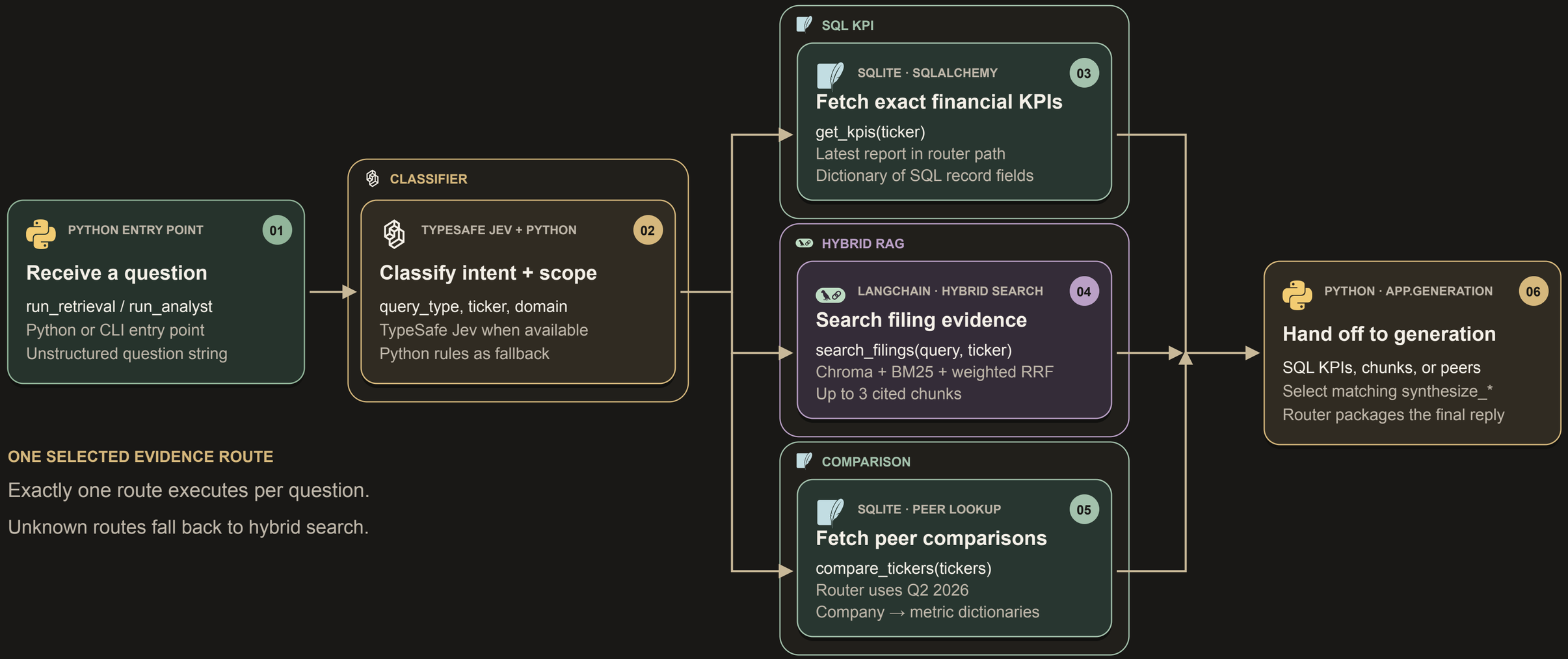
The two lanes show shared inputs; the current code attempts vector indexing first, then BM25, sequentially.

Indexing errors are logged and swallowed. COMPLETED alone does not prove both indexes are ready.

SQLite KPIs are already committed before either index is attempted.

One question, three evidence routes

Classify intent, select one retrieval tool, and pass its evidence to the matching generation function.



ONE SELECTED EVIDENCE ROUTE

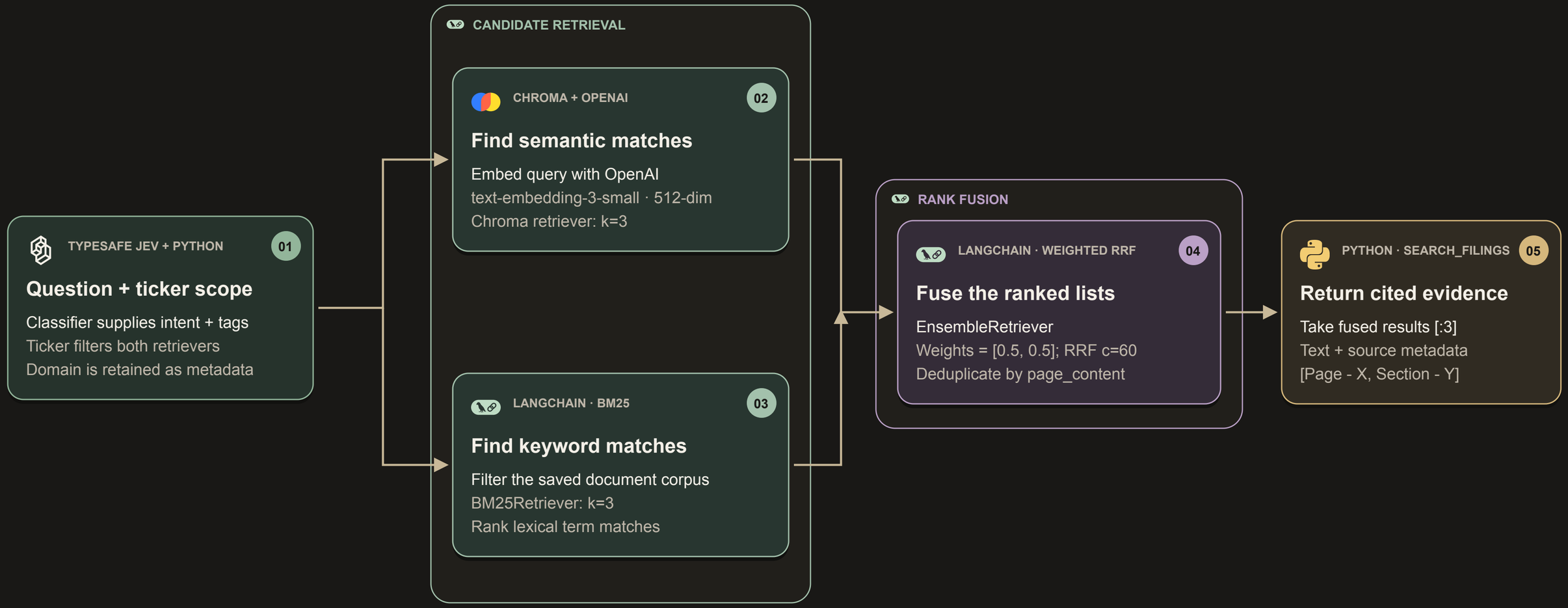
Exactly one route executes per question.

Unknown routes fall back to hybrid search.

Current interface: backend Python / CLI. Flutter chat still generates local responses; no chat HTTP or SSE route is wired.

Hybrid retrieval: meaning + exact keywords

The analyst route asks each retriever for 3 candidates, fuses rankings, and returns up to 3 evidence chunks.



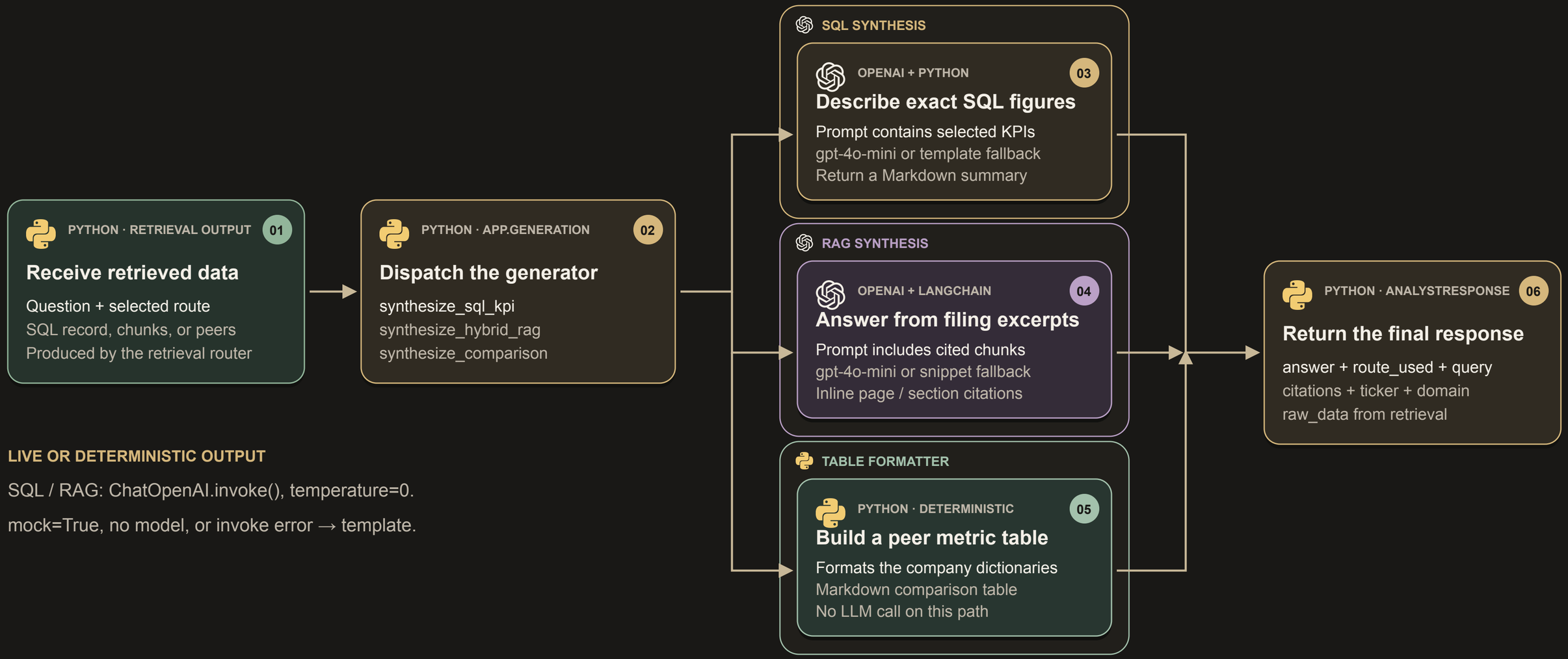
FILTERS & FALLBACK

Normal hybrid path filters ticker only. On engine.search failure, BM25 fallback also applies financial_domain if provided.

No cross-encoder reranker is called. Direct HybridEngine.search defaults to k=5; the analyst router overrides it to k=3.

From retrieved evidence to an answer

Each retrieval route selects its own synthesis function; peer comparison always uses a deterministic table.



LIVE OR DETERMINISTIC OUTPUT

SQL / RAG: ChatOpenAI.invoke(), temperature=0.

mock=True, no model, or invoke error → template.

The functions return answer strings. The retrieval router adds metadata; no token streaming or citation validator runs here.

RAG generation: keep evidence in the prompt

The live narrative path uses retrieved excerpts and inline citation labels to guide a grounded answer.

PROMPT & MODEL

ANSWER & RESPONSE

PYTHON · RETRIEVAL 01

Receive cited chunks

query + retrieved chunk payloads
text, page, domain, citation
Empty input → no-evidence notice

LANGCHAIN · MESSAGES 02

Assemble the prompt

Question + Source [citation]
Join excerpts with separators
Use ONLY supplied evidence

OPENAI 03

Generate the answer

ChatOpenAI: gpt-4o-mini
temperature=0.0; invoke()
System + Human messages

PYTHON · GENERATION 04

Return model content

resp.content becomes answer
No post-generation validation
Complete string, no SSE tokens

PYTHON · ROUTER 05

Package the response

Router adds the retrieved chunks
route_used = hybrid_rag
AnalystResponse to Python caller

CITATION INSTRUCTION

Every key factual claim should cite its excerpt:
[Page - X, Section - Y]
The prompt requests this format; code does not verify
citation correctness or detect unsupported claims.

DETERMINISTIC FALLBACK

Used for mock mode, missing model, or invoke failure.
Format the first 280 characters of each excerpt
as a bullet, retaining its citation label.
This is excerpt formatting, not a live LLM answer.

The comparison path bypasses this LLM flow entirely. SQL synthesis uses a separate exact-figures prompt.

The backend result is available through Python / CLI; Flutter's current chat widget is not connected to these functions.